

LAB 01

AIM: Installation of Python 3(Anaconda) for lab preparation

THEORY:

Anaconda is a free and open-source distribution of the Python and R programming languages designed specifically for data science, machine learning, and scientific computing. It simplifies package management and deployment by bundling a large number of popular data science libraries and tools into a single installation.

STEPS:

1. Download the Installer

- Go to the official Anaconda website:
<https://www.anaconda.com/products/distribution>
- Choose your operating system: Windows, macOS, or Linux.
- Download the Python 3.x version installer.

2. Run the Installer

- Double-click the downloaded .exe file.
- Click Next to start installation.
- Read and agree to the license terms.
- Choose whether to install just for your user or all users.
- Select installation location.
- Check Add Anaconda to my PATH environment variable.
- Click Install and wait for the process to finish.
- Click Finish.

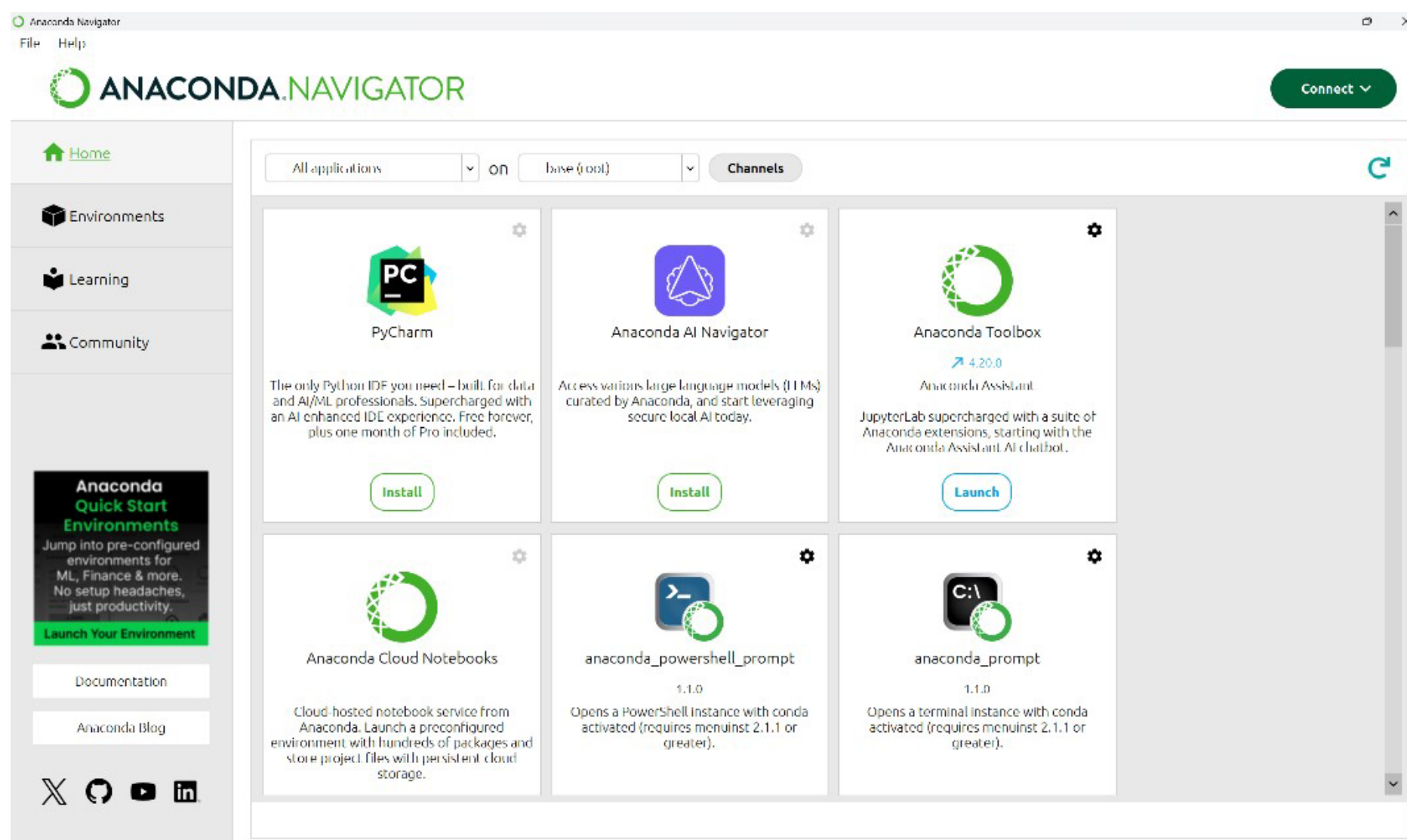
3. Verify Installation

- Open Command Prompt
- Type: `conda --version`
- If you see a version number, conda is installed correctly.
- Check Python version: `python --version` • It should display Python 3.x from Anaconda.

4. Launch Anaconda Navigator

- Search for and open Anaconda Navigator.
- This is a GUI to manage environments and launch apps like Jupyter Notebook.

OUTPUT:



LAB 02

AIM: Demonstration of the concept of data preprocessing and cleaning using Python libraries

THEORY:

Data preprocessing is the initial step in the data analysis or machine learning pipeline that involves transforming raw data into a clean, organized, and usable format. It prepares the data so that models can learn from it effectively. Data cleaning is a crucial part of preprocessing that focuses on identifying and correcting errors or inconsistencies in the data to improve its quality.

CODE:

```
import pandas as pd
import numpy as np
from scipy import stats
from sklearn.preprocessing import MinMaxScaler
```

```
df = pd.read_excel(r"C:\Users\ishan\Desktop\DWDM
LAB\messy_nepal_dataset.xlsx")
missing_before = df.isnull().sum()
df_filled = df.copy()
for col in df_filled.select_dtypes(include=np.number).columns:
    df_filled[col].fillna(df_filled[col].mean(), inplace=True)
for col in df_filled.select_dtypes(include='object').columns:
    df_filled[col].fillna(df_filled[col].mode()[0], inplace=True)
missing_after = df_filled.isnull().sum()
```

```
missing_comparison = pd.concat([missing_before, missing_after], axis=1)
missing_comparison.columns = ['Missing Before', 'Missing After']
print("\nMissing values (Before vs After filling):")
print(missing_comparison)
```

```

for col in df_filled.select_dtypes(include='object').columns:
    df_filled[col] = df_filled[col].str.replace('/', '', regex=False).str.strip()

if 'Mobile_No' in df_filled.columns:
    df_filled['Mobile_No'] = df_filled['Mobile_No'].astype(str).str.replace(r'\D', '', regex=True)

df_filled.drop_duplicates(inplace=True) print(f"\nData shape
after removing duplicates: {df_filled.shape}")

numeric_cols = df_filled.select_dtypes(include=np.number).columns
z_scores = stats.zscore(df_filled[numeric_cols])
abs_z_scores = np.abs(z_scores) filtered_entries =
(abs_z_scores < 3).all(axis=1) df_clean =
df_filled[filtered_entries] print(f"\nData shape after
removing outliers: {df_clean.shape}")

scaler = MinMaxScaler() df_clean[numeric_cols] =
scaler.fit_transform(df_clean[numeric_cols])

df_clean.to_excel("cleaned_nepal_dataset.xlsx", index=False)
print("\n Cleaned data saved to 'cleaned_nepal_dataset.xlsx'")
print(" Ishan Shrestha 28181")

```

OUTPUT:

Missing values (Before vs After filling):

	Missing Before	Missing After
User_ID	0	0
Full_Name	0	0
Email_ID	2	0
Mobile_No	2	0
Province	1	0
Subscription	1	0
Contact_Pref	2	0
Notes	2	0

Data shape after removing duplicates: (10, 8)

Data shape after removing outliers: (10, 8)

Cleaned data saved to 'cleaned_nepal_dataset.xlsx'

Ishan Shrestha 28181

messy_nepal_dataset.xlsx

A	B	C	D	E	F	G	H
User_ID	Full_Name	Email_ID	Mobile_No	Province	Subscription	Contact_Pref	Notes
2001	Anil Gurung	anil@abc.com	980-1122334	Bagmati	Active	Yes	Verified
2002	Sushma /Thapa	sushmathapa@gmail.com	984/556/7788	Lumbini	Inactive	No	Needs update
2003	Nabin KC		981 777 3333	Province 1	Na	N	
2004	/Rita Lama	rita@domain.com	9800000000		Active	Y	Valid
2005	Manish Rai	manish@xyz.com	9841234567	Gandaki	Y		
2006	Sabina Neupane			Madhesh	Inactive	N	Old contact
2007	Rajendra Shahi	raj.shahi@mail.com	986-232-1111	Karnali	Active	Yes	Confirmed
2008	/Pooja Khadka	pooja.k@abc.com		Sudurpashchim	Active	No	Recheck email
2009	Kabita Joshi	kabita@gmail.com	980-4444-555	Bagmati			-
2010	Prakash /Tamang	prakash@tmg.com	984-555-4321	Gandaki	Y	Yes	Verified

cleaned_nepal_dataset.xlsx

A	B	C	D	E	F	G	H
User_ID	Full_Name	Email_ID	Mobile_No	Province	Subscription	Contact_Pref	Notes
0	Anil Gurung	anil@abc.com	9801122334	Bagmati	Active	Yes	Verified
0.111111111	Sushma Thapa	sushmathapa@gmail.com	9845567788	Lumbini	Inactive	No	Needs update
0.222222222	Nabin KC	anil@abc.com	9817773333	Province 1	Na	N	Verified
0.333333333	Rita Lama	rita@domain.com	9800000000	Bagmati	Active	Y	Valid
0.444444444	Manish Rai	manish@xyz.com	9841234567	Gandaki	Y	Yes	Verified
0.555555556	Sabina Neupane	anil@abc.com	9801122334	Madhesh	Inactive	N	Old contact
0.666666667	Rajendra Shahi	raj.shahi@mail.com	9862321111	Karnali	Active	Yes	Confirmed
0.777777778	Pooja Khadka	pooja.k@abc.com	9801122334	Sudurpashchim	Active	No	Recheck email
0.888888889	Kabita Joshi	kabita@gmail.com	9804444555	Bagmati	Active	Yes	-
1	Prakash Tamang	prakash@tmg.com	9845554321	Gandaki	Y	Yes	Verified

LAB 03

AIM: Implementation of Apriori algorithm and generation of association rules

THEORY:

The Apriori algorithm is a classic method used in data mining to identify frequent itemsets and generate association rules from transactional datasets. It works by first scanning the dataset to find individual items that meet a minimum support threshold and then extending those items to larger and larger itemsets, as long as they remain frequent. Once frequent itemsets are discovered, the algorithm generates association rules, implications of the form “ if item A is purchased, item B is also likely to be purchased” based on confidence and lift metrics. This approach is widely used in market basket analysis, helping retailers understand customer purchasing behavior and optimize product placement and promotions.

CODE:


```

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules dataset = [ ['milk', 'bread',
'butter'], ['bread', 'butter'], ['milk', 'bread'], ['milk', 'butter'], ['bread'], ['milk',
'bread', 'butter'], ['milk', 'bread'] ]
from mlxtend.preprocessing import
TransactionEncoder te = TransactionEncoder()
te_data = te.fit(dataset).transform(dataset) df
= pd.DataFrame(te_data,
columns=te.columns_)
print("Transaction Data:")
print(df) frequent_itemsets = apriori(df, min_support=0.3, use_colnames=True)
print("\nFrequent Itemsets:") print(frequent_itemsets) rules =
association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)
rules = rules[['antecedents', 'consequents', 'support', 'confidence']]
print("\nAssociation Rules:") print(rules) print("Ishan Shrestha 28181")

```

OUTPUT:

```

Transaction Data:
   bread  butter  milk
0   True    True   True
1   True    True  False
2   True   False   True
3  False    True   True
4   True   False  False
5   True    True   True
6   True   False   True

Frequent Itemsets:
   support  itemsets
0  0.857143  (bread)
1  0.571429  (butter)
2  0.714286  (milk)
3  0.428571 (butter, bread)
4  0.571429 (bread, milk)
5  0.428571 (butter, milk)

Association Rules:
   antecedents consequents  support  confidence
0  (butter)      (bread)  0.428571      0.75
1  (milk)        (bread)  0.571429      0.80
2  (butter)      (milk)   0.428571      0.75
Ishan Shrestha 28181

```

LAB 04

AIM: Implementation of FP-Growth algorithm

THEORY:

The FP-Growth (Frequent Pattern Growth) algorithm is an efficient and scalable method for mining frequent itemsets from transactional datasets. Unlike the Apriori algorithm, which generates and tests candidate itemsets repeatedly, FP-Growth uses a compact data structure called the FP-Tree (Frequent Pattern Tree) to store itemset information in a compressed form.

This significantly reduces the number of database scans and computational time, especially with large datasets. FP-Growth works by building the tree from transactions and then recursively extracting frequent patterns without generating candidate sets. It is widely used in market basket analysis and association rule mining due to its speed and memory efficiency.

CODE:

```
import pandas as pd from mlxtend.frequent_patterns import
fpgrowth, association_rules from mlxtend.preprocessing
import TransactionEncoder
dataset = [ ['milk', 'bread', 'butter'], ['bread', 'butter'], ['milk', 'bread'], ['milk', 'butter'], ['bread'], ['milk',
'bread', 'butter'], ['milk', 'bread'] ]
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset) df = pd.DataFrame(te_ary,
columns=te.columns_) print(" Transactions:") print(df) frequent_itemsets =
fpgrowth(df, min_support=0.3, use_colnames=True) print("\nFrequent Itemsets
(FP-Growth):") print(frequent_itemsets) rules =
association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)
rules = rules[['antecedents', 'consequents', 'support', 'confidence']]
print("\nAssociation Rules:") print(rules) print("Ishan Shrestha 28181")
```

OUTPUT:

```
Transactions:
   bread  butter  milk
0   True    True   True
1   True    True  False
2   True   False   True
3  False    True   True
4   True   False  False
5   True    True   True
6   True   False   True

Frequent Itemsets (FP-Growth):
   support  itemsets
0  0.857143    (bread)
1  0.714286    (milk)
2  0.571429    (butter)
3  0.571429  (bread, milk)
4  0.428571  (butter, milk)
5  0.428571  (butter, bread)

Association Rules:
   antecedents  consequents  support  confidence
0      (milk)    (bread)  0.571429      0.80
1    (butter)    (milk)  0.428571      0.75
2    (butter)    (bread)  0.428571      0.75
Ishan Shrestha 28181
```

LAB 05

AIM: Demonstration of classification using the ID3 algorithm

THEORY:

The ID3 (Iterative Dichotomiser 3) algorithm is a popular method for constructing decision trees used in classification problems. It builds the tree by selecting the attribute that offers the highest information gain, effectively reducing uncertainty in the data. The process begins at the root and splits the dataset based on the attribute that best separates the data according to entropy, a measure of impurity. This splitting continues recursively until the data is perfectly classified or no attributes remain. ID3 is simple and efficient, making it useful in domains like medical diagnosis, marketing analysis, and pattern recognition. However, it works best with categorical data and can overfit if the tree grows too deep without pruning.

CODE:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier,
plot_tree import matplotlib.pyplot as plt

data = {
    'Age': ['<=30', '<=30', '31... 40', '>40', '>40', '>40', '31... 40',
    '<=30', '<=30', '>40', '<=30', '31... 40', '31... 40', '>40'],
    'Income': ['High', 'High', 'High', 'Medium', 'Low', 'Low', 'Low',
    'Medium', 'Low', 'Medium', 'Medium', 'Medium', 'High', 'Medium'],
    'Student': ['No', 'No', 'No', 'No', 'Yes', 'Yes', 'Yes',
    'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No'],
    'CreditRating': ['Fair', 'Excellent', 'Fair', 'Fair', 'Fair', 'Excellent', 'Excellent',
    'Fair', 'Fair', 'Fair', 'Excellent', 'Excellent', 'Fair', 'Excellent'],
    'BuysComputer': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes',
    'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No']
}

df = pd.DataFrame(data)
encoders = {} for col in
df.columns: le =
LabelEncoder() df[col] =
le.fit_transform(df[col])
encoders[col] = le
```



```

X = df.drop('BuysComputer', axis=1)
y = df['BuysComputer']

model = DecisionTreeClassifier(criterion='entropy')
model.fit(X, y)

plt.figure(figsize=(12, 8)) plot_tree(model, feature_names=X.columns,
class_names=encoders['BuysComputer'].classes_, filled=True) plt.title("Decision Tree
for Buying Computer (ID3)") plt.show()

sample_input = {
    'Age': ['<=30'],
    'Income': ['High'],
    'Student': ['Yes'],
    'CreditRating': ['Fair']
}
sample_df = pd.DataFrame(sample_input)

for col in sample_df.columns:
    sample_df[col] = encoders[col].transform(sample_df[col])

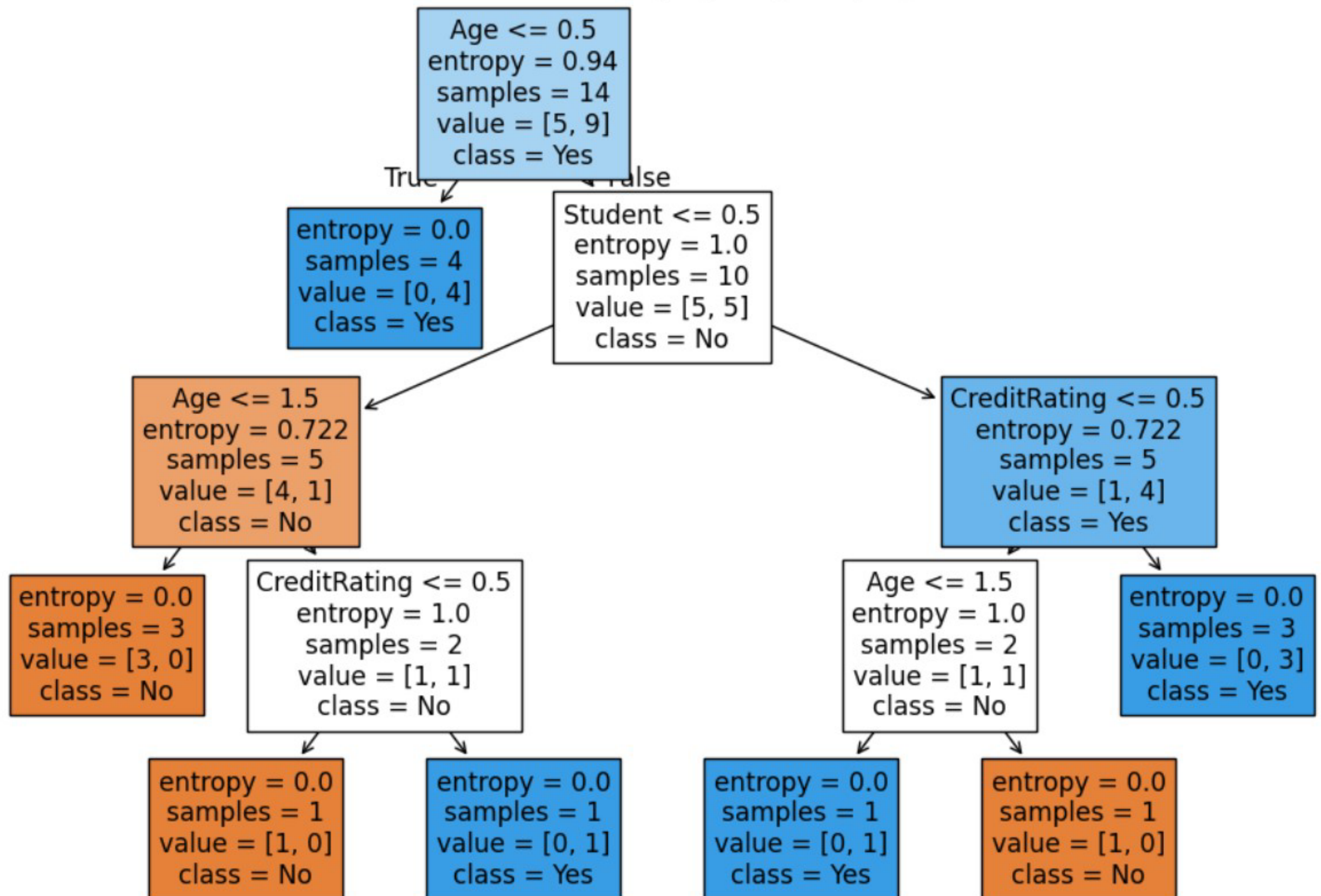
pred = model.predict(sample_df)[0] result =
encoders['BuysComputer'].inverse_transform([pred])[0]

print("Prediction for sample input:", result)
print("Ishan Shrestha 28181")

```

OUTPUT:

Decision Tree for Buying Computer (ID3)



Prediction for sample input: Yes
Ishan Shrestha 28181

AIM:

THEORY:

LAB 06

Demonstration of classification using the Bayesian algorithm

Bayesian classification is a probabilistic approach to machine learning based on Bayes' Theorem, which calculates the likelihood of a class given some input data. The most commonly used Bayesian classifier is the Naive Bayes classifier, which assumes that features are independent given the class label – an assumption that simplifies computation. It works by calculating the probability of each class and selecting the one with the highest posterior probability. Bayesian classifiers are fast, easy to implement, and effective for high-dimensional data, though they may struggle with feature correlations due to the independence assumption.

The foundation of Bayesian classification:

$$P(C | X) = \frac{P(X | C) \cdot P(C)}{P(X)}$$

Given feature vector $X = (x_1, x_2, \dots, x_n)$, the **Naive Bayes** assumption simplifies the likelihood:

$$P(X | C) = \prod_{i=1}^n P(x_i | C)$$

CODE:

```
import pandas as pd from
sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import
train_test_split from sklearn.metrics import
accuracy_score

data = {
    'Age': ['<=30', '<=30', '31... 40', '>40', '>40', '>40', '31... 40',
'<=30', '<=30', '>40', '<=30', '31... 40', '31... 40', '>40'],
    'Income': ['High', 'High', 'High', 'Medium', 'Low', 'Low', 'Low',
'Medium', 'Low', 'Medium', 'Medium', 'Medium', 'High', 'Medium'],
    'Student': ['No', 'No', 'No', 'No', 'Yes', 'Yes', 'Yes',
'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No'],
    'CreditRating': ['Fair', 'Excellent', 'Fair', 'Fair', 'Fair', 'Excellent', 'Excellent',
'Fair', 'Fair', 'Fair', 'Excellent', 'Excellent', 'Fair', 'Excellent']
```

```

        'BuysComputer': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes',
                        'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No']
    }

df = pd.DataFrame(data)
encoders = {}
for column in df.columns:
    le = LabelEncoder()
    df[column] = le.fit_transform(df[column])
    encoders[column] = le

X = df.drop('BuysComputer', axis=1)
y = df['BuysComputer']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", round(accuracy * 100, 2), "%")

sample = {
    'Age': ['<=30'],
    'Income': ['High'],
    'Student': ['Yes'],
    'CreditRating': ['Fair']
}
sample_df = pd.DataFrame(sample)

for col in sample_df.columns:
    sample_df[col] = encoders[col].transform(sample_df[col])

sample_pred = model.predict(sample_df)
result = encoders['BuysComputer'].inverse_transform(sample_pred)[0]

print("Prediction for sample input:", result)
print("Ishan Shrestha 28181")

```


AIM:

THEORY:

OUTPUT:

```
Accuracy: 60.0 %  
Prediction for sample input: Yes  
Ishan Shrestha 28181
```

LAB 07

Demonstration of classification using the SVM algorithm

The Support Vector Machine (SVM) algorithm is a powerful supervised learning model used for classification and regression tasks. It works by finding the optimal hyperplane that best separates data points of different classes in a high-dimensional space. The goal is to maximize the margin, which is the distance between the hyperplane and the nearest data points from each class, known as support vectors. By using different kernel functions, SVMs can efficiently handle non-linear classification problems by transforming the input space into higher dimensions. SVM is especially effective in cases where the number of features is large and the data is clearly separable, making it popular in fields like image recognition, bioinformatics, and text classification. To construct decision boundary or classifier:

$$f(x) = \sum_{i=1}^n \alpha_i y_i (x_i^T x) + b$$

CODE:

```
import pandas as pd from
sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import
train_test_split
from sklearn.svm import SVC from
sklearn.metrics import accuracy_score

data = {
    'Age': ['<=30', '<=30', '31... 40', '>40', '>40', '>40', '31... 40',
'<=30', '<=30', '>40', '<=30', '31... 40', '31... 40', '>40'],
    'Income': ['High', 'High', 'High', 'Medium', 'Low', 'Low', 'Low',
'Medium', 'Low', 'Medium', 'Medium', 'Medium', 'High', 'Medium'],
    'Student': ['No', 'No', 'No', 'No', 'Yes', 'Yes', 'Yes',
'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No'],
    'CreditRating': ['Fair', 'Excellent', 'Fair', 'Fair', 'Fair', 'Excellent', 'Excellent',
'Fair', 'Fair', 'Fair', 'Excellent', 'Excellent', 'Fair', 'Excellent'],
    'BuysComputer': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes',
'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No']
}

df = pd.DataFrame(data)

encoders = {} for column in df.columns:
le = LabelEncoder() df[column] =
le.fit_transform(df[column])
encoders[column] = le

X = df.drop('BuysComputer', axis=1)
y = df['BuysComputer']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = SVC(kernel='linear')
model.fit(X_train, y_train) y_pred =
model.predict(X_test)
```

AIM:

THEORY:

```
# Evaluate accuracy accuracy =
accuracy_score(y_test, y_pred)
print("Accuracy:", round(accuracy * 100, 2), "%")

sample = {
    'Age': ['<=30'],
    'Income': ['High'],
    'Student': ['Yes'],
    'CreditRating': ['Fair']
}
sample_df = pd.DataFrame(sample)

for col in sample_df.columns:
    sample_df[col] = encoders[col].transform(sample_df[col])

sample_pred = model.predict(sample_df) result =
encoders['BuysComputer'].inverse_transform(sample_pred)[0]

print("Prediction for sample input:", result)
print("Ishan Shrestha 28181")
```

OUTPUT:

```
Accuracy: 60.0 %
Prediction for sample input: Yes
Ishan Shrestha 28181
```


LAB 08

Implementation of Linear Regression algorithm

Linear Regression is a supervised learning algorithm used for predicting a continuous output based on one or more input features. It assumes a linear relationship between the independent variables and the dependent variable. The goal is to find the best-fitting straight line (also called the regression line) through the data points.

Formula: $y=wx+b$

Where, y : predicted value(dependent variable) x : input feature(independent variable)

w : slope of line

b : bias

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

data = {
    'EngineSize': [1.0, 1.2, 1.4, 1.6, 1.8, 2.0, 2.2, 2.4, 2.6, 2.8],
    'MPG': [35, 33, 30, 28, 27, 25, 23, 22, 20, 18]
}
df = pd.DataFrame(data)
```

AIM:

THEORY:

```
X = df[['EngineSize']]
```

```
y = df['MPG']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Actual vs Predicted:") result_df = pd.DataFrame({'Engine Size (L)': X_test['EngineSize'],  
'Actual MPG': y_test, 'Predicted MPG': y_pred})
```

```
print(result_df.to_string(index=False))
```

```
print("\nR2 Score:", round(r2_score(y_test, y_pred), 4)) print("Mean  
Squared Error:", round(mean_squared_error(y_test, y_pred), 2))
```

```
new_engine = pd.DataFrame({'EngineSize': [1.9]})
```

```
new_pred = model.predict(new_engine)[0]
```

```
print(f"\nPredicted MPG for engine size 1.9L: {round(new_pred, 2)}")
```

```
print("Ishan Shrestha 28181")
```

```
plt.figure(figsize=(8, 5))
```

```
plt.scatter(X, y, color='blue', label='Actual Data') plt.plot(X, model.predict(X),  
color='red', linewidth=2, label='Regression Line') plt.scatter(new_engine,  
new_pred, color='green', label='Prediction (1.9L)', zorder=5) plt.xlabel('Engine  
Size (L)') plt.ylabel('Miles Per Gallon (MPG)') plt.title('Linear Regression:  
Engine Size vs MPG') plt.legend() plt.grid(True)  
plt.show()
```

OUTPUT:

Actual vs Predicted:

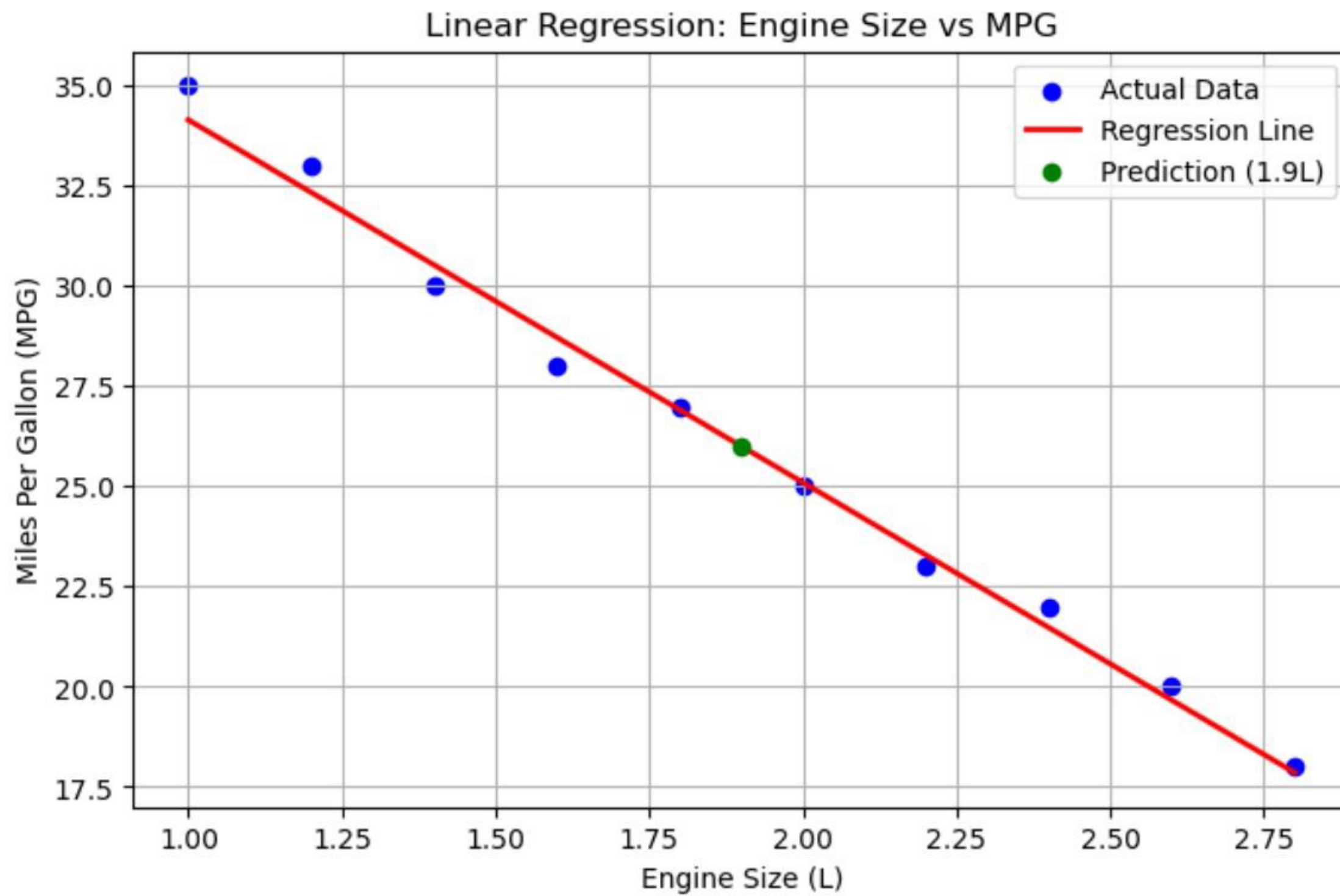
Engine Size (L)	Actual MPG	Predicted MPG
2.6	20	19.663793
1.2	33	32.336207

R^2 Score: 0.9934

Mean Squared Error: 0.28

Predicted MPG for engine size 1.9L: 26.0

Ishan Shrestha 28181



LAB 09

Implementation of Multiple Linear Regression algorithm

Multiple Linear Regression is a statistical technique used to model the relationship between a single dependent variable and two or more independent (predictor) variables. It is an extension of simple linear regression, which only considers one independent variable. The goal of multiple linear regression is to fit a linear equation to observed data, enabling us to

AIM:

THEORY:

predict the value of the dependent variable based on the values of the independent variables.

The general form of the model is:

$$y = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

where, y is the predicted outcome
 x' s are the independent variables
 w' s are their corresponding weights
 b is the bias

CODE:

```
import pandas as pd
import numpy as np
from sklearn.
```

```
linear_model import LinearRegression from
```

```
sklearn.model_selection import train_test_split from
```

```
sklearn.metrics import r2_score, mean_squared_error
```

```
data = {
```

```
    'Size': [0.8, 1.0, 1.2, 1.5, 1.7, 2.0, 2.2, 2.5, 2.7, 3.0],    # in 1000 sq.ft
```

```
    'Bedrooms': [2, 3, 2, 4, 3, 4, 5, 4, 5, 6],
```

```
    'Age': [10, 15, 12, 8, 20, 5, 7, 3, 10, 2],    # years
```

```
    'Price': [150, 180, 200, 240, 260, 300, 320, 360, 380, 400]    # in $1000s }
```

```
df = pd.DataFrame(data)
```

```
X = df[['Size', 'Bedrooms', 'Age']]
```

```
y = df['Price']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
result = pd.DataFrame({
```

```
    'Size (1000 sq.ft)': X_test['Size'],
```

```
    'Bedrooms': X_test['Bedrooms'],
```

```
    'Age (years)': X_test['Age'],
```



```

    'Actual Price ($1000s)': y_test,
    'Predicted Price ($1000s)': np.round(y_pred, 2)
})

print("Actual vs Predicted Prices:\n")
print(result.to_string(index=False)) print("\nR2 Score:",
round(r2_score(y_test, y_pred), 4)) print("Mean Squared Error:",
round(mean_squared_error(y_test, y_pred), 2))

new_input = pd.DataFrame({
    'Size': [1.8],
    'Bedrooms': [3],
    'Age': [7]
})

new_prediction = model.predict(new_input)[0] print(f"\nPredicted
Price for new house: ${round(new_prediction, 2)}k") print("Ishan
Shrestha 28181")

```

OUTPUT:

Actual vs Predicted Prices:

Size (1000 sq.ft)	Bedrooms	Age (years)	Actual Price (\$1000s)	Predicted Price (\$1000s)
2.7	5	10	380	375.19
1.0	3	15	180	176.88
2.0	4	5	300	293.92

R² Score: 0.9966

Mean Squared Error: 23.28

Predicted Price for new house: \$271.85k

Ishan Shrestha 28181

AIM:

THEORY:

LAB 10

clustering using the K-Means algorithm

K-Means clustering is an unsupervised learning algorithm that groups data into K clusters based on similarity. It begins by randomly selecting K centroids, then assigns each data point to the nearest centroid. The centroids are updated as the mean of their assigned points, and this process repeats until the centroids stabilize. K-Means aims to minimize the distance within clusters and maximize separation between them. It is fast and widely used for tasks like customer segmentation, pattern recognition, and image compression, though it can be sensitive to outliers and initial centroid positions.

CODE:

```
import warnings
warnings.filterwarnings("ignore", message="KMeans is known to have a
memory leak on Windows with MKL")
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

data = {
    'Domain': [1.8, 2.2, 2.0, 6.7, 7.1, 6.9, 11.5, 12.0, 12.4, 12.2],
    'Range': [2.1, 1.9, 2.3, 6.8, 7.0, 7.2, 2.0, 1.8, 2.4, 2.1]
}

df = pd.DataFrame(data)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(df)

plt.figure(figsize=(8, 6))
plt.scatter(df['Domain'], df['Range'], c=df['Cluster'],
            cmap='Set1', s=100, edgecolor='k')
plt.scatter(kmeans.cluster_centers_[0, 0],
            kmeans.cluster_centers_[0, 1], c='black', marker='X', s=200,
            label='Centroids')
plt.title("K-Means Clustering (Domain vs Range) - 10 Data
Points")
plt.xlabel("Domain")
plt.ylabel("Range")
plt.legend()
plt.grid(True)
plt.show()

print("Cluster Centers (Centroids):\n", kmeans.cluster_centers_)
print("\nFinal Clustered Data Table:")
print(df.to_string(index=False))
print("Ishan Shrestha 28181")
```


AIM: Demonstration of

THEORY:

OUTPUT:

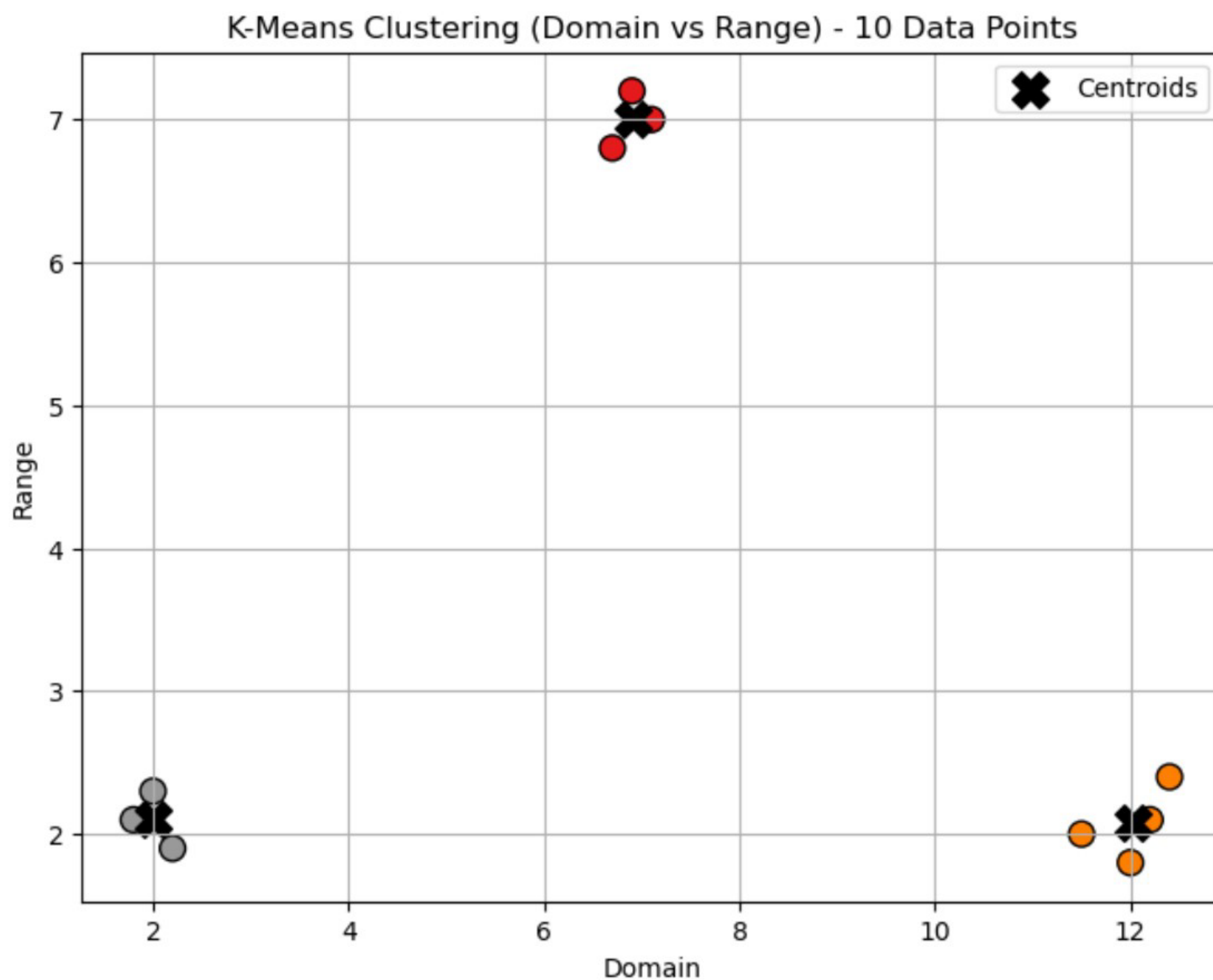
Cluster Centers (Centroids):

```
[[ 6.9    7.   ]
 [12.025  2.075]
 [ 2.     2.1  ]]
```

Final Clustered Data Table:

Domain	Range	Cluster
1.8	2.1	2
2.2	1.9	2
2.0	2.3	2
6.7	6.8	0
7.1	7.0	0
6.9	7.2	0
11.5	2.0	1
12.0	1.8	1
12.4	2.4	1
12.2	2.1	1

Ishan Shrestha 28181



LAB 11

clustering using the K-Medoids algorithm

K-Medoids is a clustering algorithm similar to K-Means, but instead of using the mean of data points to define the cluster center, it uses actual data points called medoids. A medoid is the most centrally located point in a cluster, minimizing the total distance to all other points in the same cluster. The algorithm starts by randomly selecting K medoids, assigning each data point to the nearest medoid, and then updating the medoids by checking if swapping any point with a medoid reduces the overall cost (sum of distances). This process repeats until no further improvement is possible. K-Medoids is more robust to noise and outliers than K-Means, since it does not use means which can be skewed. It's especially useful when working with non-Euclidean distance measures or when outliers heavily influence clustering quality.

CODE:

AIM: Demonstration of

THEORY:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import pairwise_distances
import random

data = {
    'Domain': [1.8, 2.2, 2.0, 6.7, 7.1, 6.9, 11.5, 12.0, 12.4, 12.2],
    'Range': [2.1, 1.9, 2.3, 6.8, 7.0, 7.2, 2.0, 1.8, 2.4, 2.1]
}
df = pd.DataFrame(data)
X = df[['Domain', 'Range']].to_numpy()
k = 3
max_iter = 100
random.seed(42)
medoid_indices = random.sample(range(len(X)), k)
medoids = X[medoid_indices]
for _ in range(max_iter):
    # Step 2: Assign points to the nearest medoid
    distances = pairwise_distances(X, medoids)
    clusters = np.argmin(distances, axis=1)
    new_medoids = np.copy(medoids)
    for i in range(k):
        members = X[clusters == i]
        if len(members) == 0:
            continue
        cost = pairwise_distances(members, members).sum(axis=1)
        new_medoids[i] = members[np.argmin(cost)]
    if np.all(medoids == new_medoids):
        break
medoids = new_medoids
df['Cluster'] = clusters
plt.figure(figsize=(8, 6))
plt.scatter(df['Domain'], df['Range'], c=df['Cluster'], cmap='Set2', s=100,
            edgecolor='black')
plt.scatter(medoids[:, 0], medoids[:, 1], c='red', marker='X', s=200,
            label='Medoids')
plt.title("K-Medoids Clustering (Domain vs Range)")
plt.xlabel("Domain")
plt.ylabel("Range")
plt.legend()
plt.grid(True)
plt.show()
print("Medoid Coordinates:")
for i, medoid in enumerate(medoids):
```

```

print(f"Cluster {i}: Domain = {medoid[0]}, Range = {medoid[1]}")
print("\nClustered Data Table:") print(df.to_string(index=False))
print("Ishan Shrestha 28181")

```

OUTPUT:

```

Medoid Coordinates:
Cluster 0: Domain = 2.2, Range = 1.9
Cluster 1: Domain = 1.8, Range = 2.1
Cluster 2: Domain = 11.5, Range = 2.0

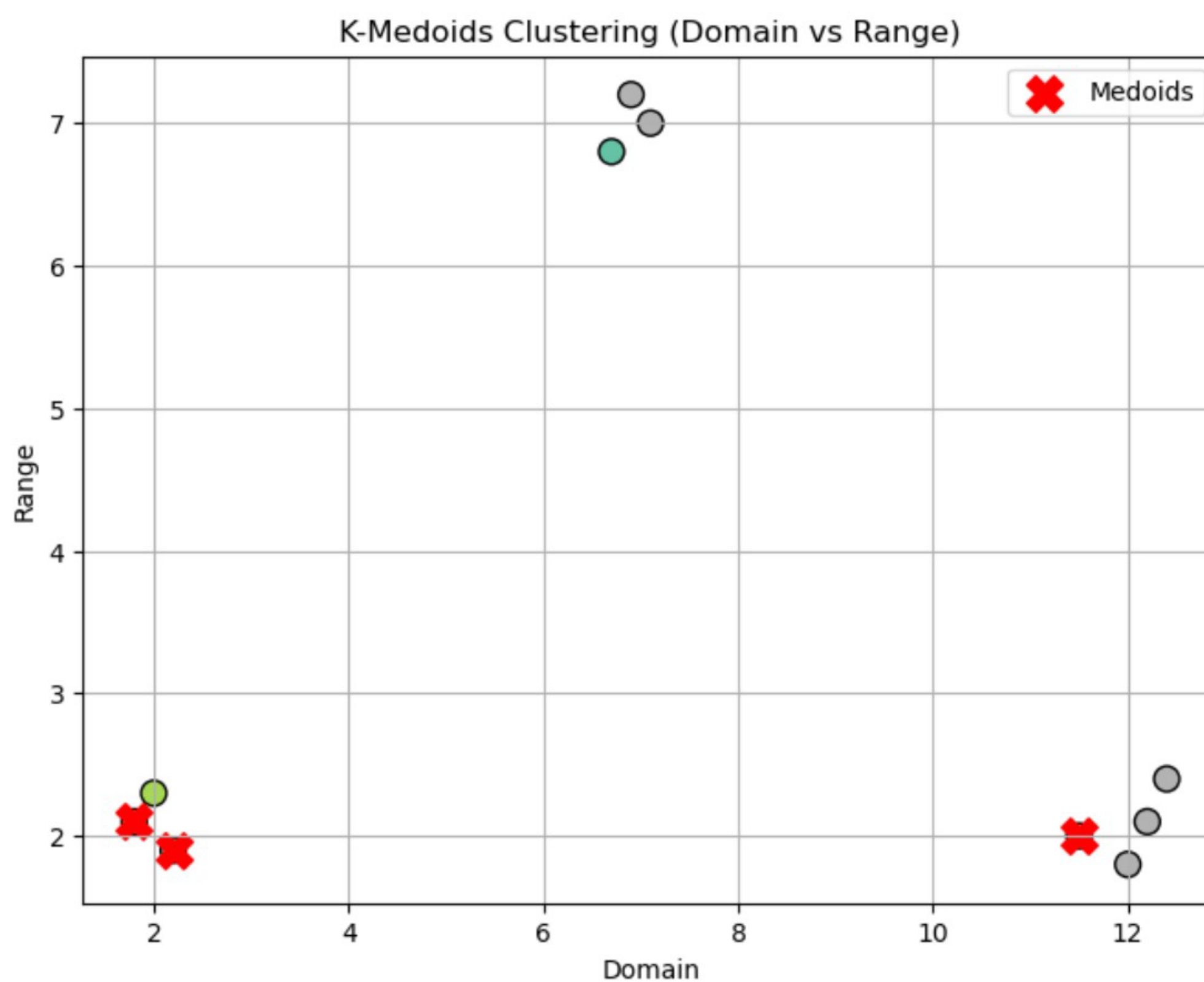
```

```

Clustered Data Table:
Domain  Range  Cluster
    1.8    2.1        1
    2.2    1.9        0
    2.0    2.3        1
    6.7    6.8        0
    7.1    7.0        2
    6.9    7.2        2
   11.5    2.0        2
   12.0    1.8        2
   12.4    2.4        2
   12.2    2.1        2

```

```
Ishan Shrestha 28181
```



AIM: Demonstration of

THEORY:

LAB 12

clustering using the Mini-Batch K Means algorithm

Mini-Batch K-Means is a faster and more scalable variation of the traditional K-Means clustering algorithm designed to handle large datasets efficiently. Instead of using the entire dataset to update the cluster centroids in each iteration, Mini-Batch K-Means uses small, random subsets (mini-batches) of the data to update the centroids. This significantly reduces computation time while still converging to a solution close to that of standard K-Means. The algorithm repeatedly samples mini-batches, assigns their points to the nearest centroids, and updates the centroids incrementally. Mini-Batch K-Means trades off some accuracy for speed, making it ideal for big data applications where quick clustering is more critical than perfect precision.

CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import MiniBatchKMeans
```

```
data = {
    'Domain': [1.8, 2.2, 2.0, 6.7, 7.1, 6.9, 11.5, 12.0, 12.4, 12.2],
    'Range': [2.1, 1.9, 2.3, 6.8, 7.0, 7.2, 2.0, 1.8, 2.4, 2.1]
}
```

```
df = pd.DataFrame(data)
```

```
X = df[['Domain', 'Range']]
k = 3
mbk = MiniBatchKMeans(n_clusters=k, random_state=42, batch_size=5)
df['Cluster'] = mbk.fit_predict(X)
```

```
plt.figure(figsize=(8, 6))
plt.scatter(df['Domain'], df['Range'], c=df['Cluster'],
            cmap='Accent', s=100, edgecolor='black')
plt.scatter(mbk.cluster_centers_[0, 0], mbk.cluster_centers_[0, 1],
            c='red', marker='X', s=200, label='Centroids')
plt.title("Mini-Batch K-Means Clustering (Domain vs Range)")
plt.xlabel("Domain")
plt.ylabel("Range")
plt.grid(True)
plt.legend()
plt.show()
```

```
print("Cluster Centers:")
for i, center in enumerate(mbk.cluster_centers_):
```



```
print(f"Cluster {i}: Domain = {center[0]:.2f}, Range = {center[1]:.2f}")
```

```
print("\nClustered Data Table:")
```

```
print(df.to_string(index=False))
```

```
print("Ishan Shrestha 28181")
```

OUTPUT:

Cluster Centers:

Cluster 0: Domain = 12.02, Range = 2.09

Cluster 1: Domain = 6.90, Range = 6.98

Cluster 2: Domain = 1.97, Range = 2.12

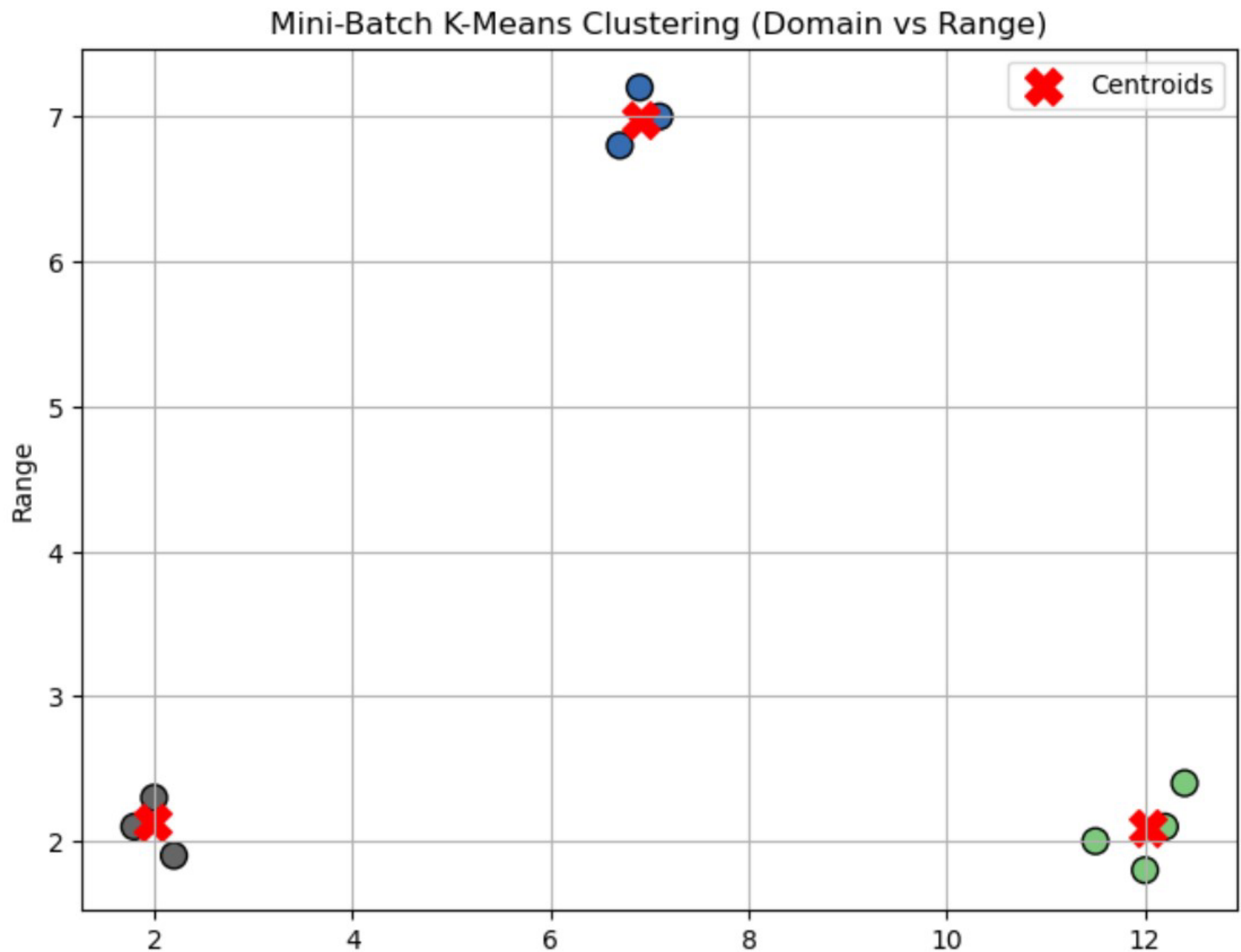
Clustered Data Table:

Domain	Range	Cluster
1.8	2.1	2
2.2	1.9	2
2.0	2.3	2
6.7	6.8	1
7.1	7.0	1
6.9	7.2	1
11.5	2.0	0
12.0	1.8	0
12.4	2.4	0
12.2	2.1	0

Ishan Shrestha 28181

AIM: Demonstration of

THEORY:



LAB 13

clustering using the Agglomerative algorithm

Agglomerative clustering is a type of hierarchical clustering algorithm that builds clusters bottom-up. It starts by treating each data point as its own cluster and then iteratively merges the two closest clusters based on a chosen similarity or distance metric. This merging process continues until all points belong to a single cluster or a stopping criterion (like a desired number of clusters) is reached. The method produces a tree-like structure called a dendrogram, which shows the order and distances at which clusters were merged. Agglomerative clustering is useful when the number of clusters is unknown beforehand and is widely applied in biology, social network analysis, and document clustering. It is flexible in choosing different linkage criteria (e.g., single, complete, average), which affect how cluster distances are calculated.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import
AgglomerativeClustering
from
scipy.cluster.hierarchy import dendrogram, linkage
import seaborn as sns

data = {
    'Domain': [1.8, 2.2, 2.0, 6.7, 7.1, 6.9, 11.5, 12.0, 12.4, 12.2],
    'Range': [2.1, 1.9, 2.3, 6.8, 7.0, 7.2, 2.0, 1.8, 2.4, 2.1]
}
df = pd.DataFrame(data)

X = df[['Domain', 'Range']]

agg = AgglomerativeClustering(n_clusters=3, linkage='ward')
df['Cluster'] = agg.fit_predict(X)

plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='Domain', y='Range', hue='Cluster', palette='Set1',
s=100, edgecolor='black')
plt.title("Agglomerative Clustering (Domain vs Range)")
plt.xlabel("Domain")
plt.ylabel("Range")
plt.legend(title='Cluster')
plt.grid(True)
plt.show()

plt.figure(figsize=(10, 6))
linked = linkage(X, method='ward')
dendrogram(linked, labels=df.index.tolist(), orientation='top', distance_sort='ascending', show_leaf_counts=True)
plt.title("Dendrogram for Hierarchical Clustering")
plt.xlabel("Sample Index")
plt.ylabel("Distance")
plt.grid(True)
plt.tight_layout()
plt.show()
print("Clustered Data Table:")
print(df.to_string(index=False))
print("Ishan Shrestha 28181")
```

OUTPUT:

AIM: Demonstration of

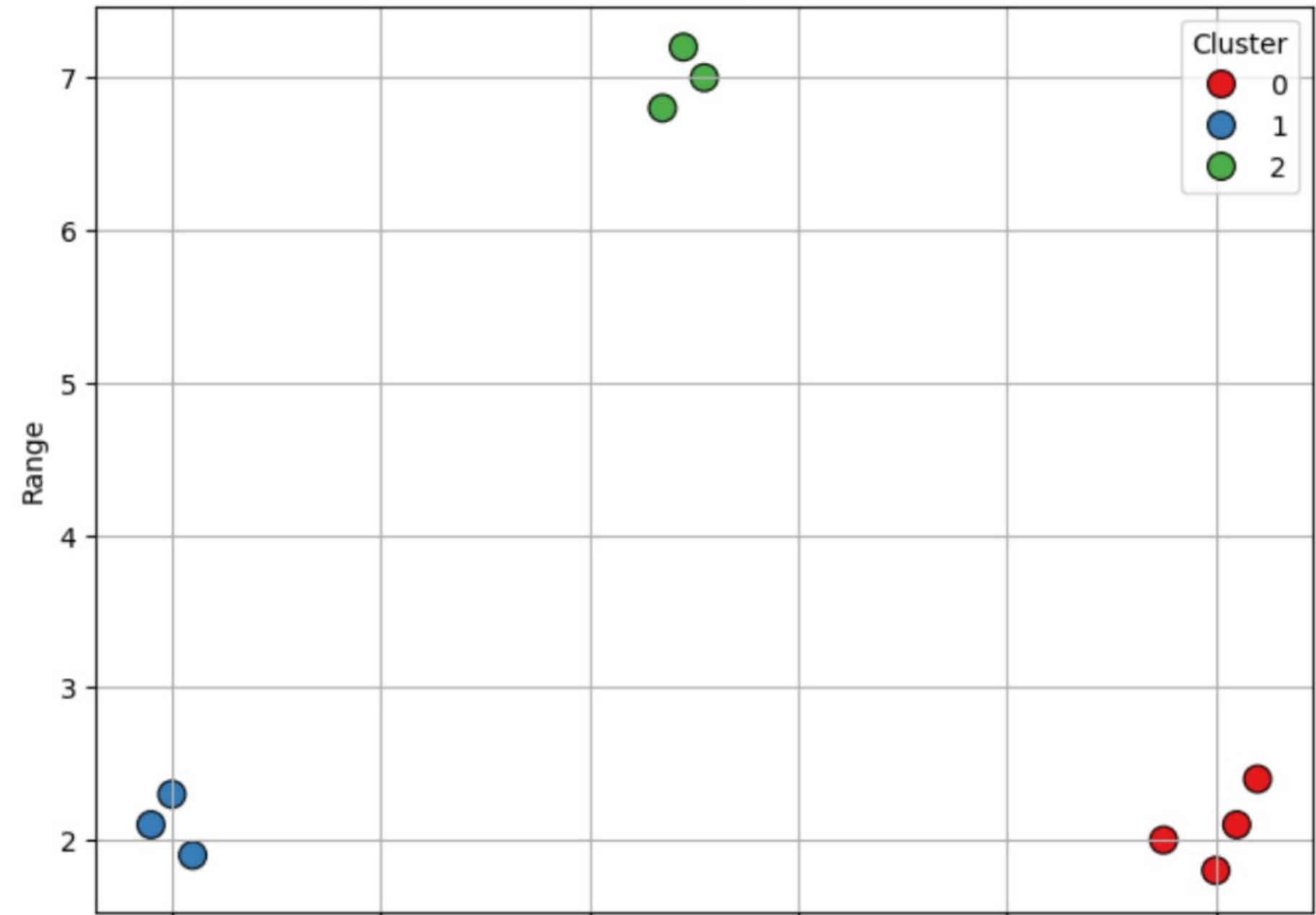
THEORY:

Clustered Data Table:

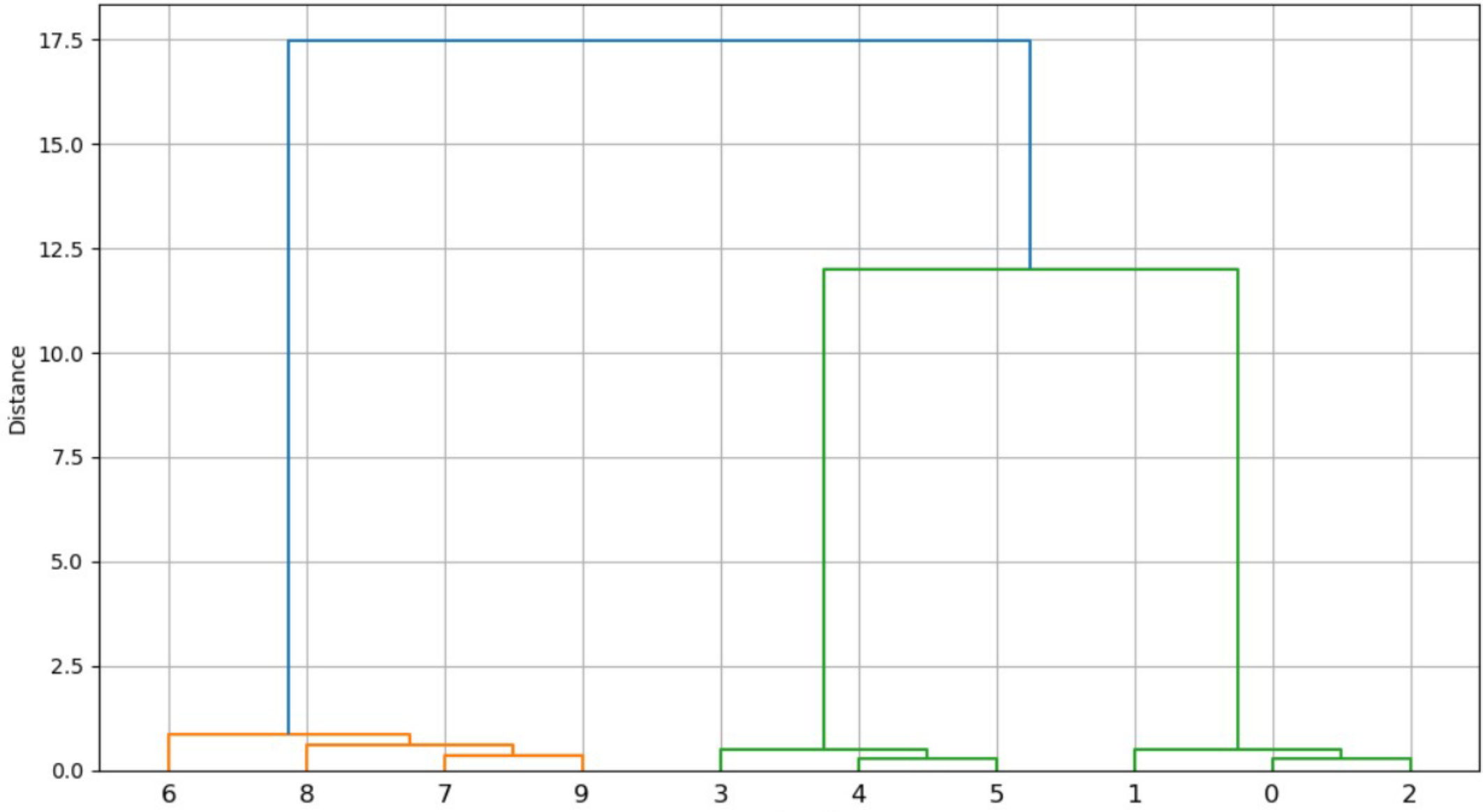
Domain	Range	Cluster
1.8	2.1	1
2.2	1.9	1
2.0	2.3	1
6.7	6.8	2
7.1	7.0	2
6.9	7.2	2
11.5	2.0	0
12.0	1.8	0
12.4	2.4	0
12.2	2.1	0

Ishan Shrestha 28181

Agglomerative Clustering (Domain vs Range)



Dendrogram for Hierarchical Clustering



LAB 14

clustering using the DBSCAN algorithm

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a powerful clustering algorithm that groups together points that are closely packed and marks points in low-density regions as outliers or noise. Unlike K-Means, it does not require specifying the number of clusters in advance.

DBSCAN relies on two parameters:

- ϵ (epsilon): the maximum distance between two points to be considered neighbors.
- MinPts: the minimum number of neighboring points required to form a dense region.

The algorithm classifies points into:

- Core points: have at least MinPts neighbors within ϵ .
- Border points: are within ϵ of a core point but don't have enough neighbors to be core points.
- Noise points: are neither core nor border.

DBSCAN starts from an unvisited point, checks its neighborhood, and expands the cluster if it is a core point. It is ideal for discovering clusters of arbitrary shape and for datasets with noise, making it useful in geospatial data analysis, anomaly detection, and image processing.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
import seaborn as sns
```

```
data = {
    'Domain': [1.8, 2.2, 2.0, 6.7, 7.1, 6.9, 11.5, 12.0, 12.4, 8.5],
    'Range': [2.1, 1.9, 2.3, 6.8, 7.0, 7.2, 2.0, 1.8, 2.4, 9.0]
}
df = pd.DataFrame(data)
```

```
X = df[['Domain', 'Range']]
```

```
dbscan = DBSCAN(eps=0.8, min_samples=2)
df['Cluster'] = dbscan.fit_predict(X)
```

```
plt.figure(figsize=(8, 6))
palette = sns.color_palette('Set1',
n_colors=len(set(df['Cluster'])))
```


AIM: Demonstration of

THEORY:

```
sns.scatterplot(data=df, x='Domain', y='Range', hue='Cluster', palette=palette,
s=100, edgecolor='black') plt.title("DBSCAN Clustering (Domain vs Range)")
plt.xlabel("Domain") plt.ylabel("Range") plt.legend(title='Cluster (- 1 = Noise)') plt.grid(True)
plt.tight_layout()
plt.show()
```

```
print("Clustered Data Table:")
```

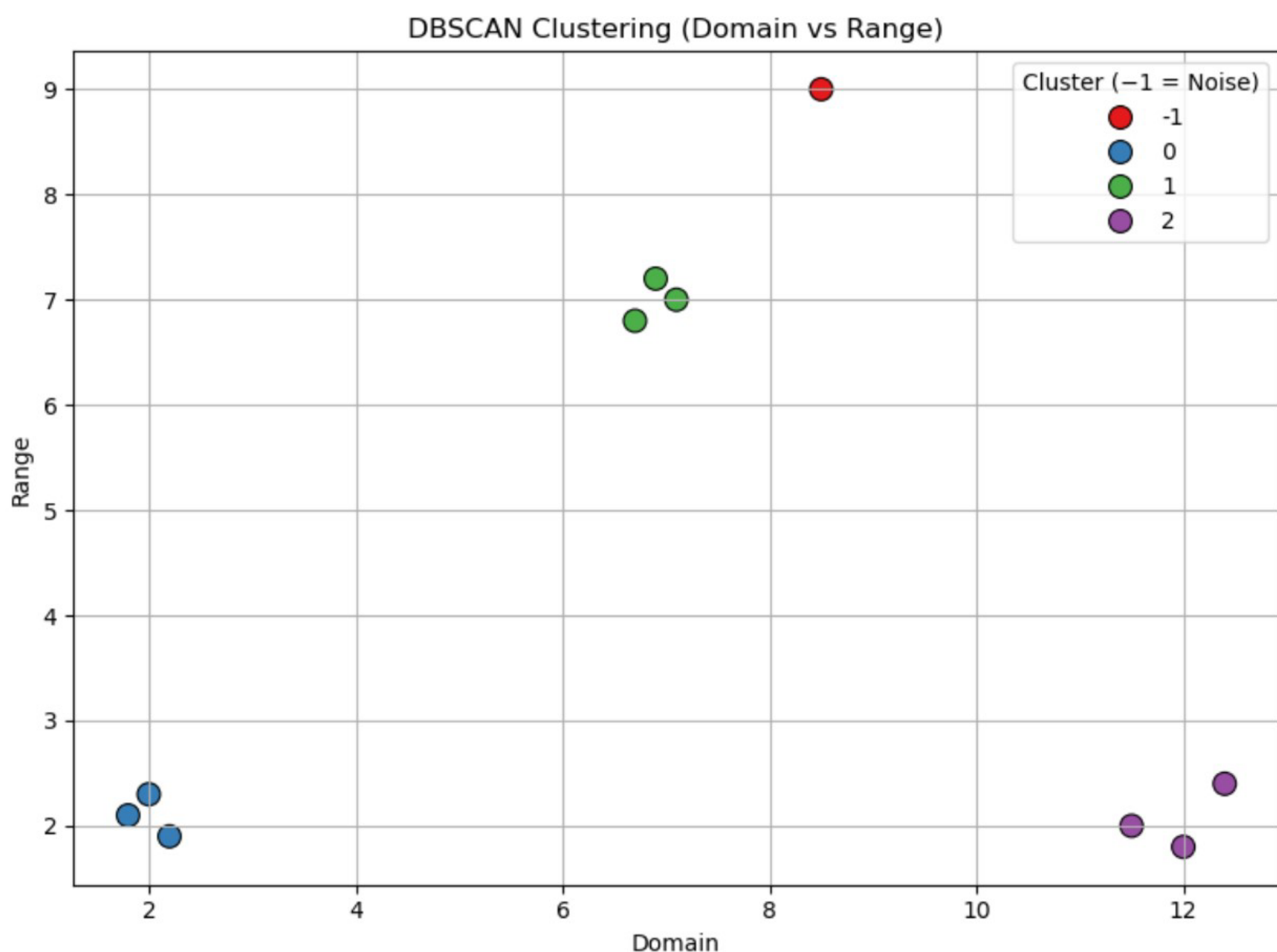
```
print(df.to_string(index=False))
```

```
print("Ishan Shrestha 28181") OUTPUT:
```

Clustered Data Table:

Domain	Range	Cluster
1.8	2.1	0
2.2	1.9	0
2.0	2.3	0
6.7	6.8	1
7.1	7.0	1
6.9	7.2	1
11.5	2.0	2
12.0	1.8	2
12.4	2.4	2
8.5	9.0	-1

Ishan Shrestha 28181

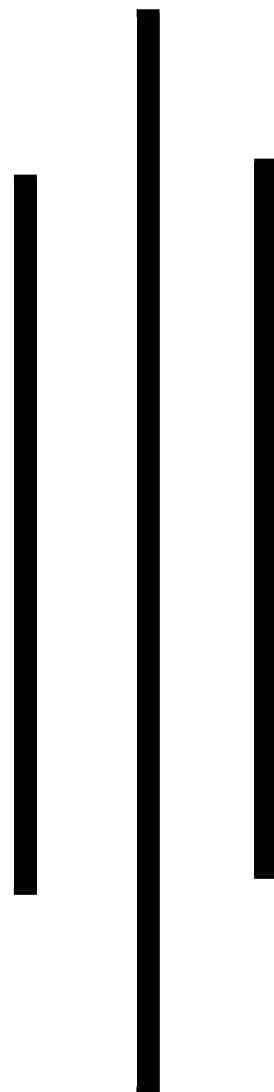




TRIBHUVAN UNIVERSITY

AMRIT SCIENCE CAMPUS

Lainchaur, Kathmandu



B.SC.CSIT SEVENTH SEMESTER

Lab Report of

Data Warehousing and Data Mining

Submitted By:

Name : Ishan Shrestha

Symbol No. : 28181/078

Submitted to:

Mr. Nabaraj Bahadur Negi

Department of Computer

Section : A

Science and Information
Technology

TABLE OF CONTENTS

Lab No.	Topic	Page No.
1.	Installation of Python 3(Anaconda) for lab preparation	1
2.	Demonstration of the concept of data preprocessing and cleaning using Python libraries	2
3.	Implementation of Apriori algorithm and generation of association rules	4
4.	Implementation of FP-Growth algorithm	5
5.	Demonstration of classification using the ID3 algorithm	6
6.	Demonstration of classification using the Bayesian algorithm	8
7.	Demonstration of classification using the SVM algorithm	10
8.	Implementation of Linear Regression algorithm	12
9.	Implementation of Multiple Linear Regression algorithm	14
10.	Demonstration of clustering using the K-Means algorithm	16
11.	Demonstration of clustering using the K-Medoids algorithm	18
12.	Demonstration of clustering using the Mini-Batch K Means algorithm	20
13.	Demonstration of clustering using the Agglomerative algorithm	22
14.	Demonstration of clustering using the DBSCAN algorithm	24